Splat Editor: a Viewer and Editor for 3D Gaussian Splatting Scenes

Lê Anh Kiệt [a]

[a] *International University, Vietnam National University Ho Chi Minh City, Vietnam* Student ID: ITCSIU24047

P R O J E C T I N F O

Keywords:
3D Gaussian Splatting
Camera and projection
Lighting and shadows
Physics simulation
ImGui interface

A B S T R A C T

This project builds Splat Editor, a program to view and edit 3D Gaussian Splatting (3DGS) scenes in Python and OpenGL 3.3. The program implements the 3DGS rendering pipeline itself: it reads the PLY file, projects ellipsoids to ellipses, sorts by depth and blends with alpha. On top of this, the project uses the six technique groups of the course: transformations, camera, texture, lighting, animation and interaction. The user can select, delete, undo and save the file.

1. Introduction and gaussian theory

1.1. Introduction

3D Gaussian Splatting (3DGS) shows a 3D scene with millions of soft ellipsoids, called gaussians [2]. The scene is trained from photos taken around an object (Fig. 1). The resulting `.ply` file always has floating gaussians. So we need a program to view and clean the file. SuperSplat [3] does this in the browser. I wrote a similar editor in Python and OpenGL 3.3.

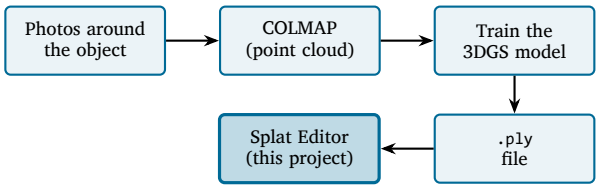


Fig. 1. The workflow from photos to Splat Editor. COLMAP [1] finds the camera positions.

The project has two parts. Part one is the 3DGS rendering pipeline, from the file to the shader. Part two is the six technique groups of the course (Table 1).

Table 1. Technique groups and sections.

Group	Main content	Section
T1	layers, matrix **M**, character skeleton	2
T2	view/projection matrices, EWA, blending, cameras	3
T3	skin texture, texture buffers, gaussian kernel	4
T4	SH colour, Phong, shadow map	5
T5	day–night, collision, character animation	6
T6	selection brush, delete/undo, ImGui, HUD	7

1.2. Input data

The input is a `.ply` file made by 3DGS training (Fig. 2). Training starts from the COLMAP point cloud and puts one gaussian at each point. Then it repeats three steps. It renders the gaussians. It compares the image with the real photo. It updates the parameters of every gaussian. It also removes useless gaussians and splits gaussians that are too large.

The file has two parts: a text header and a binary body. The header lists the properties. The body has one record per gaussian. Every value is a 32-bit float. Table 2 lists the properties. With SH of degree 3, one record has 62 values, or 248 bytes. Fig. 3 shows the first rows of a file, exported as CSV.

Table 2. Properties of one gaussian in the PLY file.

Property	Count	Meaning
`x, y, z`	3	centre μ
`nx, ny, nz`	3	normal, always 0, not used
`f_dc_0..2`	3	base colour (SH degree 0)
`f_rest_*`	45	SH coefficients of degrees 1–3
`opacity`	1	raw opacity
`scale_0..2`	3	logarithm of the size
`rot_0..3`	4	rotation quaternion

```
x,y,z,nx,ny,nz,f_dc_0,                                  f_dc_1,f_dc_2,
2.98395634,0.64003098,1.57549512,0,0,0,0.0184020139,-0.12875317,-0.452952206,
4.22914267,-0.663039207,-0.523675323,0,0,0,-0.150325835,-0.135893673,-0.325479239,
1.13421345,0.410973638,3.93819642,0,0,0,0.346005678,0.0445601828,-0.405238956,
0.604746521,2.51691461,-0.371717632,0,0,0,-0.157206044,-0.176818103,-0.256703675,
0.642261147,-1.39632988,-4.3651309,0,0,0,-0.227364123,-0.334438473,-0.433887929,
-4.2221036,-0.695849061,0.247175366,0,0,0,0.412634581,0.405996054,0.111131459,
```

```
opacity,scale_0,scale_1,scale_2,rot_0,rot_1,rot_2,rot_3
-0.0695948228,-0.0550651364,-0.0382503457,-0.0237467252,0.0131118316,0.0629977509,-0.0449319854,-0.
0.000325970963,-0.0160180721,0.00357675459,0.0473092273,0.0389139801,-0.0540050268,-0.0265164431,-0
0.0612790994,0.0957935154,-0.0631214827,0.0422011353,0.0244775601,0.0491610095,0.0470059477,0.07206
-6.13165855,-2.71730924,-2.80323267,-2.86005892,0.876973867,0.112431385,0.0801429227,-0.0182835758
-5.53735924,-2.7295742,-2.76182842,-2.74456096,0.885511041,0.101086691,-0.0186818913,-0.0949812829
-0.0379554592,0.00524016004,0.0189723969,-0.0380355604,-0.0966132209,-0.00303976983,-0.0181281436,-
```

Fig. 3. The first six gaussians of a PLY file, shown as CSV. Top: position, normal and base colour. Bottom: opacity, scale and rotation. The `f_rest` columns in between are hidden.

The program maps a binary file straight into memory. So a file with millions of gaussians opens almost at once. Fig. 4 shows a small capture of a person. The blurry blobs around the body are the floating gaussians that the editor must remove. The main test scene is a street with about 5 million gaussians (Fig. 5).

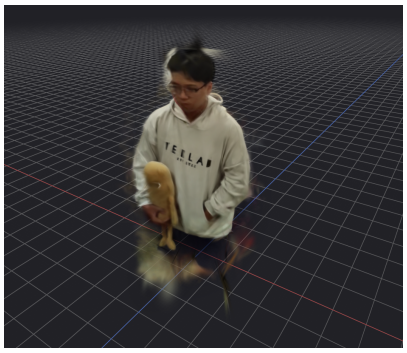


Fig. 4. A PLY file of a captured person. Floating gaussians are visible around the body and near the ground.

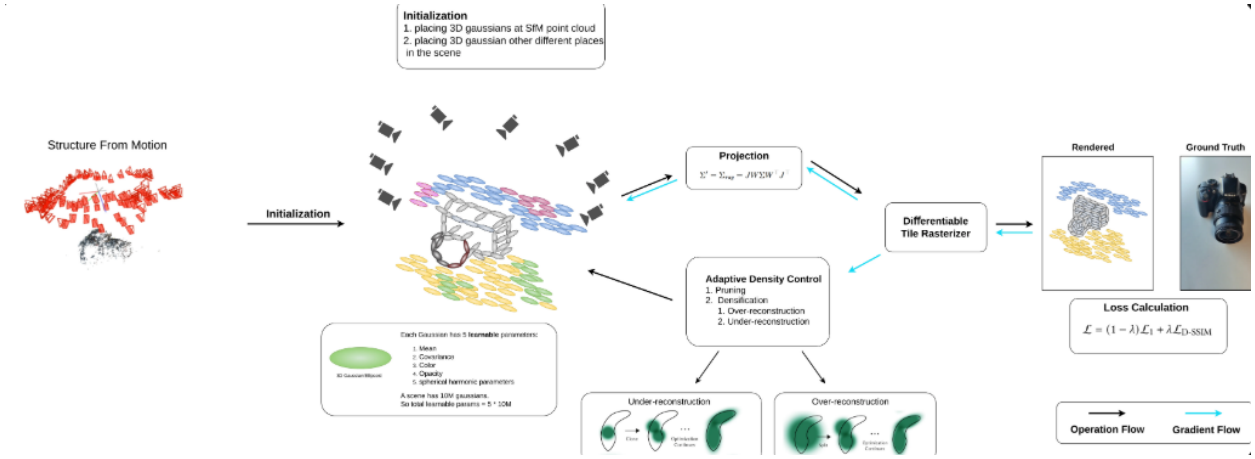


Fig. 2. How 3DGS training makes the input file [2]. The gaussians start at the structure-from-motion points. They are projected, rendered and compared with the ground-truth photo. The result of this loop is the `.ply` file.

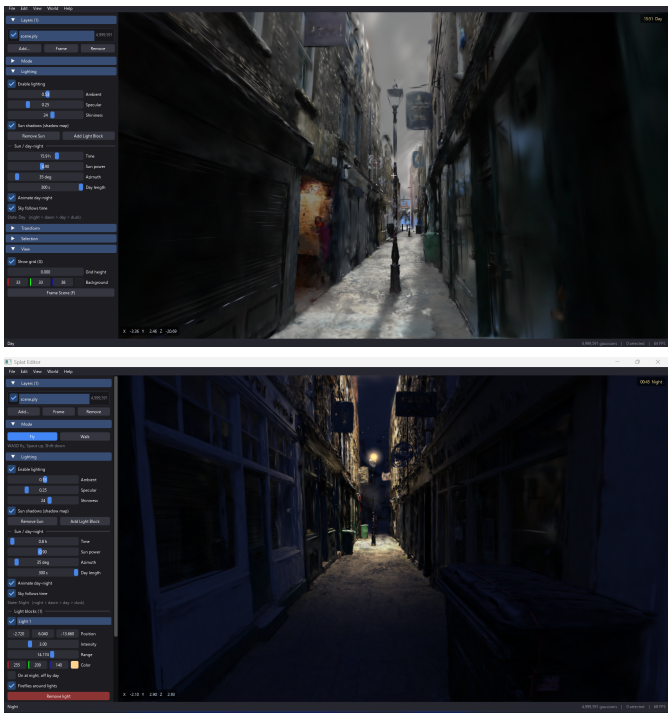


Fig. 5. A scene of about 5 million gaussians in Splat Editor, by day (top) and by night (bottom).

1.3. Gaussians and the PLY file

Each gaussian has a position $\boldsymbol{\mu}$, a colour $\mathbf{c}$, an opacity α, a size $\mathbf{s}$ and a rotation $\mathbf{q}$ (quaternion). Its density falls off smoothly from the centre. We can picture it as an ellipsoid (Fig. 6). The PLY file stores raw values. The program decodes them on load:

- Size: $s_i = \exp(\texttt{scale_i})$.
- Opacity: $\alpha = 1/(1 + e^{-\texttt{opacity}})$.
- Colour: RGB $= 0.5 + C_0\,\texttt{f_dc}$, with $C_0 \approx 0.2821$.
- Rotation: the quaternion `rot_0..3` is normalised to length 1.

On save, the program does the reverse.

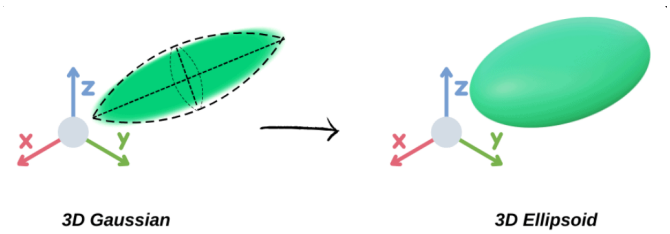


Fig. 6. A 3D gaussian is a soft blob. Its surface at one standard deviation is an ellipsoid.

The shape of a gaussian is stored in its covariance matrix $\boldsymbol{\Sigma}$. It is built from the scale matrix $\mathbf{S} = \operatorname{diag}(\mathbf{s})$ and the rotation matrix $\mathbf{R}(\mathbf{q})$:

$$\boldsymbol{\Sigma} = \mathbf{R}\,\mathbf{S}\mathbf{S}^{\top}\mathbf{R}^{\top}. \tag{1}$$

Fig. 7 gives the two matrices. Fig. 8 draws the result for the first gaussian of Fig. 3. The three sizes s_0, s_1, s_2 are the semi-axes of the ellipsoid. The quaternion turns these axes in space. Section 3.2 projects this ellipsoid to the screen.

$$R = \begin{bmatrix} 1-2(q_y^2+q_z^2) & 2(q_xq_y - q_zq_w) & 2(q_xq_z + q_yq_w) \\ 2(q_xq_y + q_zq_w) & 1-2(q_x^2+q_z^2) & 2(q_yq_z - q_xq_w) \\ 2(q_xq_z - q_yq_w) & 2(q_yq_z + q_xq_w) & 1-2(q_x^2+q_y^2) \end{bmatrix}$$

$$S = \begin{bmatrix} s_0 & 0 & 0 \\ 0 & s_1 & 0 \\ 0 & 0 & s_2 \end{bmatrix}$$

Fig. 7. The rotation matrix $\mathbf{R}$ from the quaternion (q_w, q_x, q_y, q_z) and the scale matrix $\mathbf{S}$.

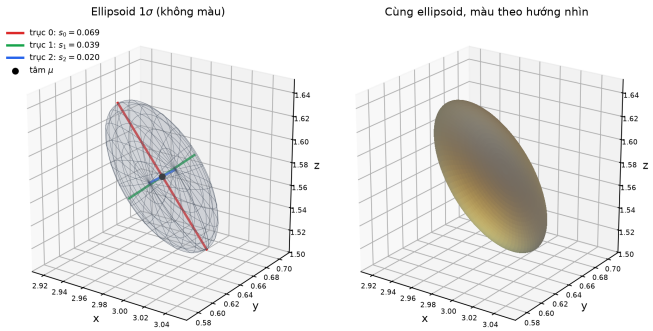
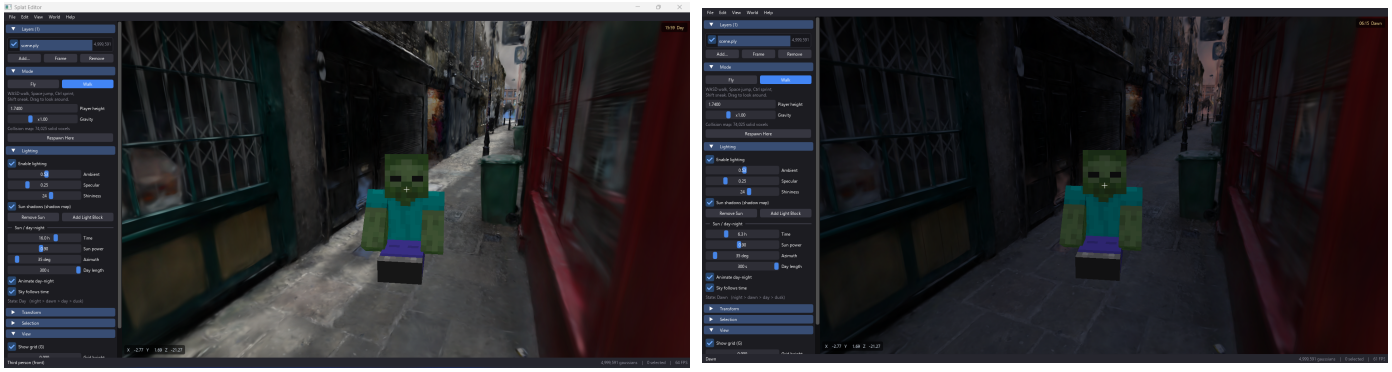


Fig. 8. One gaussian from the file. Left: the 1σ ellipsoid with its three axes and its centre $\boldsymbol{\mu}$. Right: the same ellipsoid with its view-dependent colour.



(a) Day, 15:59.

(b) Dawn, 06:15.

(c) Ambient 0.08 and a lamp.

(d) Dusk, 17:40.

Fig. 9. A scene with the sun, one point light and the character at different times of day.

2. T1: Transformations and hierarchical models

This chapter has three parts. The first two move a whole layer, the last one moves a hierarchical model.

- **Part 1, layer transformation.** Put a gaussian of a `.ply` file into the world.
- **Part 2, rotation about a point.** Rotate a layer around a chosen centre.
- **Part 3, character skeleton.** Build the matrices of the character from a hierarchy of joints.

2.1. Part 1: Layer transformation

Each `.ply` file is a layer. A layer has a translation vector $\mathbf{t}$, three Euler angles and a scale factor s. They form the matrix $\mathbf{M}$. This matrix takes a gaussian from file space to world space:

$$\mathbf{p}_w = s\mathbf{A}\mathbf{p}_f + \mathbf{t}, \qquad \mathbf{\Sigma}_w = s^2\mathbf{A}\,\mathbf{\Sigma}_f\,\mathbf{A}^\top. \tag{2}$$

Here $\mathbf{A} = \mathbf{R}_z\mathbf{R}_y\mathbf{R}_x\mathbf{F}$. $\mathbf{F} = \mathrm{diag}(-1,-1,1)$ is an optional flip for COLMAP files, whose Y axis points down. The gaussian data does not change. Only $\mathbf{M}$ changes. So moving, rotating or scaling a layer is instant. On save, the program writes the transform into every gaussian. It also rotates the SH coefficients so the view-dependent colour stays correct.

2.2. Part 2: Rotation about a point

A layer can rotate about a point $\mathbf{c}$ with a matrix $\mathbf{\Delta}$:

$$\mathbf{R}' = \mathbf{\Delta}\mathbf{R}, \qquad \mathbf{t}' = \mathbf{\Delta}(\mathbf{t} - \mathbf{c}) + \mathbf{c}. \tag{3}$$

2.3. Part 3: Character skeleton

The character appears when you press `F5`. It has six boxes: head, body, two arms and two legs. Each box has its own matrix. This matrix is the product of the joint matrices from the root down (Fig. 10). Each joint rotates about a point $\mathbf{p}$:

$$\mathbf{M}_{\text{child}} = \mathbf{M}_{\text{parent}}\,\mathbf{T}(\mathbf{p})\,\mathbf{R}\,\mathbf{T}(-\mathbf{p}). \tag{4}$$

The legs are children of the root. The arms and head are children of the body, so they lean with the body. The joint angles come from the animation (Section 6).

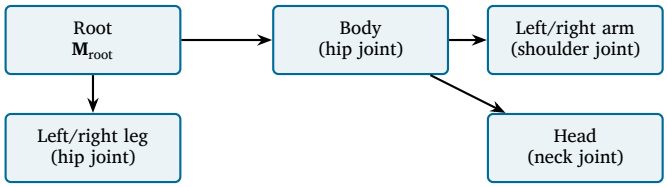


Fig. 10. The character hierarchy. Each arrow is one matrix multiplication.

3. T2: Camera and projection

A gaussian goes through four steps to become pixels on the screen [4]. Each step is one box in Fig. 11, read from left to right.

- **Part 1, viewing transformation.** Move the gaussian from object space to camera space.
- **Part 2, projective mapping.** Turn the 3D ellipsoid into a 2D ellipse.
- **Part 3, shading and integration.** Compute the opacity of each pixel and blend the gaussians together.
- **Part 4, viewport transformation.** Place the result in the viewport of the window.
- **Part 5, three cameras.** The camera gives the input of part 1.

The five parts below follow this order.

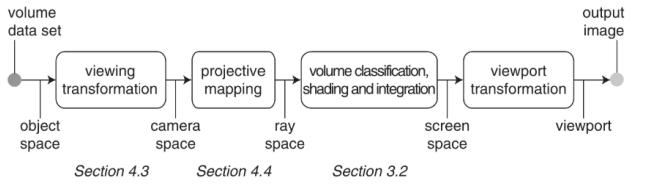


Fig. 11. The splatting pipeline of EWA [4]. The viewing transformation is **W** and the projective mapping is approximated by **J**.

3.1. Part 1: Viewing transformation

This step changes the coordinate system. After it, the camera is at the origin and looks along the $-z$ axis. The `look_at` function builds three camera axes from the eye position **e** and the target: right **r**, up **u** and forward **f**. The view matrix is

$$\mathbf{V} = \begin{pmatrix} \mathbf{r}^\top & -\mathbf{r}\cdot\mathbf{e} \\ \mathbf{u}^\top & -\mathbf{u}\cdot\mathbf{e} \\ -\mathbf{f}^\top & \mathbf{f}\cdot\mathbf{e} \\ \mathbf{0}^\top & 1 \end{pmatrix}. \tag{5}$$

The centre of the gaussian is multiplied by **V**. The result is the centre in camera space (v_x, v_y, v_z). The shape of the gaussian is only rotated. Let **W** be the rotation part of **V**, that is its top-left 3×3 block: $\mathbf{W} = \begin{pmatrix} \mathbf{r} & \mathbf{u} & -\mathbf{f} \end{pmatrix}^\top$. The ellipsoid (1) in camera space is $\mathbf{W}\boldsymbol{\Sigma}\mathbf{W}^\top$.

3.2. Part 2: Projective mapping

This step takes camera space to the image plane.

Projecting a point. The perspective matrix uses the vertical field of view φ ($k = 1/\tan(\varphi/2)$), the aspect ratio a, the near plane n and the far plane f:

$$\mathbf{P} = \begin{pmatrix} \frac{k}{a} & 0 & 0 & 0 \\ 0 & k & 0 & 0 \\ 0 & 0 & \frac{f+n}{n-f} & \frac{2fn}{n-f} \\ 0 & 0 & -1 & 0 \end{pmatrix}. \tag{6}$$

The point is multiplied by **P** and divided by c_w. The result is the normalised coordinates (n_x, n_y) in the range $[-1, 1]$. The division by c_w makes far objects look smaller. The centre of the gaussian is projected this way.

Projecting a gaussian. A gaussian is not a point. Its ellipsoid must become an ellipse on screen. The division by c_w is not linear, so we cannot multiply **P** with the ellipsoid. EWA replaces the projection by its derivative (the Jacobian) at the centre of the gaussian:

$$\mathbf{J} = \begin{pmatrix} \frac{f_{px}}{-v_z} & 0 & \frac{f_{px}\, v_x}{v_z^2} \\ 0 & \frac{f_{px}}{-v_z} & \frac{f_{px}\, v_y}{v_z^2} \\ 0 & 0 & 0 \end{pmatrix}. \tag{7}$$

Here $f_{px} = kH/2$ is the focal length in pixels, with k from (6) and H the viewport height. Joining step 1 and step 2, the 2D covariance is

$$\boldsymbol{\Sigma}' = \mathbf{J}\,\mathbf{W}\,\boldsymbol{\Sigma}\,\mathbf{W}^\top\mathbf{J}^\top. \tag{8}$$

Here $\boldsymbol{\Sigma}$ is the 3D covariance (1). **W** is the rotation part of the view matrix from step 1. **J** is the Jacobian (7). Let the top-left 2×2 block of $\boldsymbol{\Sigma}'$ be $\begin{pmatrix} a & b \\ b & c \end{pmatrix}$. Its eigenvalues are

$$\lambda_{1,2} = \frac{a+c}{2} \pm \sqrt{\left(\frac{a-c}{2}\right)^2 + b^2}. \tag{9}$$

The two semi-axes of the ellipse are proportional to $\sqrt{\lambda_1}$ and $\sqrt{\lambda_2}$. The shader draws each gaussian as a quad covering at most 3 standard deviations. It drops gaussians whose centre is behind the camera or more than 1.3 times outside the screen.

3.3. Part 3: Shading and alpha blending

This step gives each pixel a colour. For each pixel of the quad, the fragment shader finds the distance r to the centre of the ellipse, measured in standard deviations. The opacity has a bell shape:

$$a = \alpha\, e^{-r^2/2}. \tag{10}$$

The gaussians are drawn from far to near and blended:

$$\mathbf{C}_i = \mathbf{c}_i a_i + (1 - a_i)\,\mathbf{C}_{i-1}. \tag{11}$$

Here $\mathbf{C}_{i-1}$ is the colour accumulated before splat i. The result depends on the order, so we must sort by depth. The program converts depth to 16 bits and sorts it with NumPy in a background thread. Until the sort is done, it draws with the order of the previous sort. The depth test is turned off, because a transparent gaussian must not hide the gaussians behind it.

3.4. Part 4: Viewport transformation

The last step converts the normalised coordinates (n_x, n_y) of step 2 to pixels. The viewport has size $W \times H$ and corner (x_0, y_0):

$$x_{px} = x_0 + \frac{n_x+1}{2} W, \qquad y_{px} = y_0 + \frac{1-n_y}{2} H. \tag{12}$$

OpenGL does this step itself when drawing. The program also uses this formula to know which pixel a gaussian is on, for example in brush selection (Section 7).

3.5. Part 5: Three cameras

Orbit camera. The camera always looks at a point **t**. Its position uses a yaw angle θ, a pitch angle ϕ and a distance d:

$$\mathbf{e} = \mathbf{t} + d\,(\cos\phi\sin\theta,\ \sin\phi,\ \cos\phi\cos\theta). \tag{13}$$

The pitch is limited to about $\pm 89°$ so the camera does not flip.

Fly camera. The velocity does not change at once. It moves gradually to the wanted velocity $\mathbf{v}^*$, so the camera has inertia:

$$\mathbf{v} \leftarrow \mathbf{v} + (\mathbf{v}^* - \mathbf{v})\min(10\Delta t, 1), \qquad \mathbf{e} \leftarrow \mathbf{e} + \mathbf{v}\,\Delta t. \tag{14}$$

Third-person view. In walk mode, the camera pulls back behind the character. The program casts a ray from the character's eye to the camera position in the voxel map. If the ray hits a wall, the camera pulls in at once so it does not go through the wall. When the path is clear, it moves back out gradually.

4. T3: Texture mapping

The program uses textures in three ways. Each way is one subsection below.

- **Part 1, character skin.** Paint the box model of the character with an image.
- **Part 2, data as textures.** Store the gaussians in texture buffers.
- **Part 3, procedural kernel.** Compute the gaussian opacity in the shader, with no image.

4.1. Part 1: Character skin

The skin is a 64×64 image in the Minecraft cross layout. My own `load_png` function reads it. The `GL_NEAREST` filter keeps the pixels sharp. Each box face has its own 4 vertices. The UV of a vertex is its pixel position divided by 64. Each box is drawn twice: a base layer and an overlay layer. The fragment shader discards the transparent pixels of the overlay.



Fig. 12. The 64 × 64 skin used for the character (Minecraft cross layout). Each box (head, body, arms, legs) owns one cross, and the overlay (hat, jacket, sleeves) has another.

Face geometry. A box has origin (o_x, o_y) in the skin and size (w, h, d). Its centre is $\mathbf{c}$ and its overlay layer is inflated by δ ($\delta = 0$ for the base, 0.25 for body and limbs, 0.5 for the hat). A corner of a face is $\mathbf{p} = \mathbf{c} + \big(s_x(\frac{w}{2}+\delta),\, s_y(\frac{h}{2}+\delta),\, s_z(\frac{d}{2}+\delta)\big)$ with $s_x, s_y, s_z \in \{-1, +1\}$.

Pixel rectangle of each face. Each face takes a pixel rectangle (r_x, r_y, r_w, r_h) from the cross:

$$\begin{aligned} \text{front } (+Z) &: (o_x{+}d,\ o_y{+}d,\ w,\ h),\\ \text{back } (-Z) &: (o_x{+}2d{+}w,\ o_y{+}d,\ w,\ h),\\ \text{right } (+X) &: (o_x,\ o_y{+}d,\ d,\ h),\\ \text{left } (-X) &: (o_x{+}d{+}w,\ o_y{+}d,\ d,\ h),\\ \text{top } (+Y) &: (o_x{+}d,\ o_y,\ w,\ d),\\ \text{bottom } (-Y) &: (o_x{+}d{+}w,\ o_y,\ w,\ d). \end{aligned} \tag{15}$$

UV of the corners. The four corners of the face (top-left, top-right, bottom-right, bottom-left) get

$$\begin{aligned} (u, v) &= \Big(\frac{r_x + k\, r_w}{64},\ \frac{r_y + l\, r_h}{64}\Big),\\ (k, l) &\in \{(0,0), (1,0), (1,1), (0,1)\}. \end{aligned} \tag{16}$$

Sampling. At draw time the sampler returns the texel $(\lfloor 64u \rfloor, \lfloor 64v \rfloor)$ because of `GL_NEAREST`.

4.2. Part 2: Data as textures

The millions of gaussians live in *texture buffers*. The shader reads them with `texelFetch`. There are four buffers: gaussian data, selected/deleted state, SH coefficients and layer matrices. When the state or a layer matrix changes, the program rewrites only one small buffer. It does not upload the whole scene again.

4.3. Part 3: Procedural gaussian kernel

The opacity $a = \alpha e^{-r^2/2}$ (10) is computed directly in the fragment shader. No image is needed. The shadow map (Section 5) is also a texture.

5. T4: Lighting and shadows

The final colour of a gaussian is made in four parts.

- **Part 1, view-dependent colour.** Read the base colour from the SH coefficients.
- **Part 2, Phong lighting.** Add ambient, diffuse and specular light.
- **Part 3, the sun.** Move the sun and the sky colour with the hour.
- **Part 4, shadows.** Test each gaussian against a shadow map.

5.1. Part 1: View-dependent colour

The colour of a gaussian changes with the view direction $\mathbf{d}$. 3DGS stores this with spherical harmonics (SH) coefficients [2]:

$$\mathbf{c}(\mathbf{d}) = \mathrm{clip}\Big(\mathbf{c}_0 + \sum_{k=1}^{K} Y_k(\mathbf{d})\, \mathbf{h}_k,\ 0,\ 1\Big). \tag{17}$$

Here $\mathbf{c}_0$ is the base colour, $\mathbf{h}_k$ are the coefficients in the file and Y_k are the basis functions. The coefficients are in file space. So the shader rotates the view direction into that space: $\mathbf{d} = \mathrm{norm}(\mathbf{A}^\top(\mathbf{p}_w - \mathbf{e}))$. It does not have to rotate the coefficients every frame. Fig. 13 shows the idea for degrees 0 to 2. Each basis function is a pattern on the sphere. The weighted sum gives one colour for each view direction.

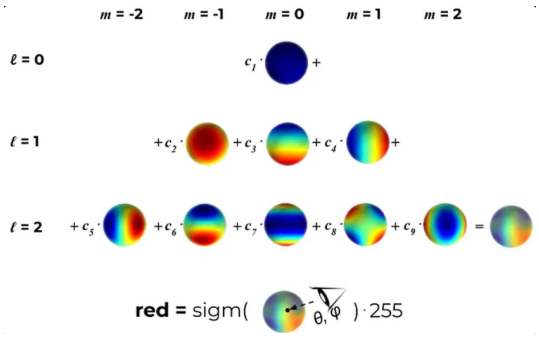


Fig. 13. Spherical harmonics up to degree 2. The nine basis functions are weighted by the coefficients $c_1..c_9$ and added. The colour is read at the view direction (θ, φ).

5.2. Part 2: Phong lighting

Lighting is computed once per gaussian. The normal $\mathbf{n}$ is the shortest axis of the ellipsoid, flipped towards the camera. The final colour uses the Phong model [5]. It has three parts (Fig. 14). Let $\mathbf{c}$ be the colour of the gaussian.

Ambient. The ambient light $\mathbf{L}_a$ reaches every gaussian equally:

$$\mathbf{I}_a = \mathbf{c}\, \mathbf{L}_a. \tag{18}$$

Diffuse. A surface that faces the light is brighter. For light j, $\mathbf{L}_j$ is its colour and $\mathbf{l}_j$ is the direction to the light:

$$\begin{aligned} \mathbf{I}_d &= \mathbf{c} \sum_j \mathbf{L}_j\, w(\mathbf{n}, \mathbf{l}_j)\, S_j,\\ w(\mathbf{n}, \mathbf{l}) &= \mathrm{clip}\Big(\frac{\mathbf{n}\cdot\mathbf{l} + 0.35}{1.35}, 0, 1\Big). \end{aligned} \tag{19}$$

The term w is a softened $\mathbf{n}\cdot\mathbf{l}$, because the estimated normal is noisy.

Specular. A highlight appears where the reflected ray $\mathbf{r}_j$ is close to the direction $\mathbf{v}$ to the camera:

$$\mathbf{I}_s = k_s \sum_j \mathbf{L}_j\, (\mathbf{r}_j \cdot \mathbf{v})_+^m\, S_j. \tag{20}$$

Here k_s is the specular strength and m is the shininess. This part does not use the colour $\mathbf{c}$. It is zero when $\mathbf{n}\cdot\mathbf{l}_j \le 0$.

Sum. The final colour is the sum of the three parts:

$$\mathbf{c}_{out} = \mathbf{I}_a + \mathbf{I}_d + \mathbf{I}_s. \tag{21}$$

S_j is the shadow factor of the sun, or the distance falloff of a point light. The scene has one sun and up to 8 point lights. Point lights cast no shadows.

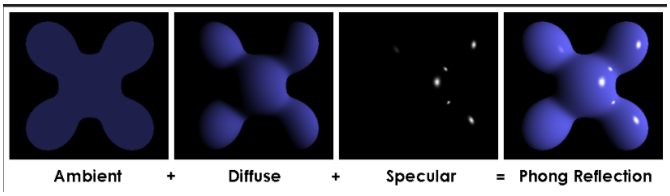


Fig. 14. The Phong model [5]: ambient, diffuse and specular parts, and their sum.

5.3. Part 3: The sun

The sun direction is computed from the hour h of the day. The sun rises at 6, is highest at 12 and sets at 18. The sky colour and the ambient light change gradually between day and night. They are slightly orange at dusk (Fig. 9).

5.4. Part 4: Shadows

The program draws the centres of the solid gaussians as points into a 2048^2 depth map seen from the sun [6]. Each gaussian is projected into this map to compare depths. If something is closer to the sun, the gaussian is in shadow. The program averages 3×3 samples to soften the shadow edge (PCF [7]):

$$S = \frac{1}{9} \sum_{i,j=-1}^{1} \left[z_s - b \le D_s(u_s + i\tau,\, v_s + j\tau) \right]. \tag{22}$$

Here (u_s, v_s, z_s) are the coordinates of the gaussian in the map, D_s is the stored depth, τ is the texel step and b is the bias. The bias stops a gaussian from shadowing itself.

6. T5: Animation and simulation

Every motion is multiplied by the frame time Δt. So the speed depends little on the machine. There are four parts, in the order of one frame.

- **Part 1, day–night cycle.** Advance the clock and move the sun.
- **Part 2, voxel collision.** Turn the gaussians into solid voxels.
- **Part 3, the player.** Walk, jump and fall on the voxels.
- **Part 4, character animation.** Compute the joint angles of the character.

6.1. Part 1: Day–night cycle

This is a state machine with 4 states: NIGHT → DAWN → DAY → DUSK. The clock runs by itself. Lights set to "on at night" turn on at dusk and off at dawn. The states start at hour 5 (DAWN), 7 (DAY), 17 (DUSK) and 19 (NIGHT). One day lasts $T_{day} = 36$ s, so the hour advances by $\Delta h = 24\,\Delta t / T_{day}$ each frame. The sun position follows the hour:

$$\theta = \frac{h - 6}{12}\pi, \qquad \epsilon = 65° \sin\theta, \;\; \mathbf{s} = \big(\cos\epsilon\cos\phi,\; \sin\epsilon,\; \cos\epsilon\sin\phi\big), \tag{23}$$

where $\phi = \phi_0 + \theta - \pi/2$ and ϕ_0 is the sun azimuth. The sun rises at 6, is highest at 12 and sets at 18. The daylight factor is $D = \text{smoothstep}(-0.08,\, 0.3,\, s_y)$. The ambient colour and the sky colour are mixed between a night value and a day value with D, and the sky gets an orange tint near the horizon.

6.2. Part 2: Collision with voxels

A gaussian has no clear surface, so direct collision is hard. The program divides space into voxels of edge v. A voxel with enough gaussians is solid. Each solid voxel is stored as one 64-bit integer:

$$\mathbf{c} = \left\lfloor \frac{\mathbf{p} - \mathbf{o}}{v} \right\rfloor, \qquad \text{key} = (c_x \ll 42) \,|\, (c_z \ll 21) \,|\, c_y. \tag{24}$$

The keys are sorted. Because c_y is in the low bits, the cells of one column sit next to each other. Finding the ground under the feet takes one `np.searchsorted` call.

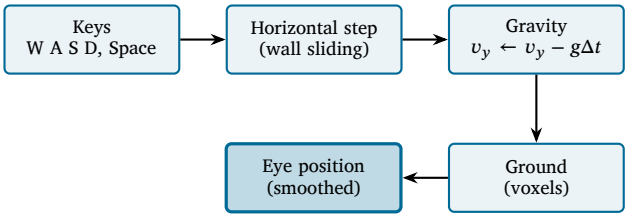


Fig. 15. One frame of walk mode.

6.3. Part 3: The player

Fig. 15 summarises one frame. The player is a cylinder. At a wall, the player slides along it. The player can climb a low step and can jump.

Gravity. When not on the ground, the program integrates with Euler steps:

$$v_y \leftarrow v_y - g\,\Delta t, \qquad y \leftarrow y + v_y\,\Delta t. \tag{25}$$

Scale and jump. All sizes and speeds scale with the player height H. The reason is that a 3DGS scene has no fixed unit. The constants are: voxel edge $v = H/8$, player radius $0.14H$, step height $0.35H$, gravity $g = 12H$, walking speed $2.5H$ per second (sprint ×1.7, sneak ×0.4). A jump reaches the height $0.75H$, so the take-off speed is

$$v_0 = \sqrt{2\,g\,(0.75H)}. \tag{26}$$

Eye smoothing. The eye height is smoothed while the player stands on the ground, so the rough gaussian surface does not shake the camera.

6.4. Part 4: Character animation

Only the joint angles are computed (in degrees); the renderer builds the matrices. Let d be the distance walked in this frame, in units of the player height. The gait phase grows with the distance, not with time, so the legs never slide on the ground:

$$\psi \leftarrow \psi + 1.75 \cdot 2\,d, \qquad \theta_{leg} = \pm A\,a\,\sin\psi, \quad \theta_{arm} = \mp A\,a\,\sin\psi, \tag{27}$$

with $A = 60°$. The leg and the opposite arm move together.

Blending states. The amplitude $a \in [0, 1]$ follows the speed with the filter $x \leftarrow x + (x^* - x)\min(1, k\,\Delta t)$ (14), with $k = 8$. The same filter, with $k = 7$, blends the other states: sit (legs −90°, body lowered), crouch (body leaning 28°) and the gestures wave, zombie and cheer. Two poses are mixed linearly, $\theta = (1 - w)\theta_a + w\,\theta_b$, so a gesture fades in and out. When standing, a small sin term moves the arms to imitate breathing.

Joint matrices. Each limb is a rotation about its joint: $\mathbf{M} = \mathbf{T}(\mathbf{j})\,\mathbf{R}(\theta)\,\mathbf{T}(-\mathbf{j})$ applied after the parent matrix, for example the arm after the torso.

7. T6: Interaction and GUI

The interaction has four parts, from the input to the screen.

- **Part 1, event handling.** Decide who receives each mouse or keyboard event.
- **Part 2, brush selection.** Select gaussians by painting on the screen.
- **Part 3, soft delete and undo.** Delete the selection and restore it.
- **Part 4, interface.** Panel, HUD and shortcuts.

7.1. Part 1: Event handling

Mouse and keyboard events go to ImGui first. If ImGui does not use them, they go to the camera, the selection brush and the shortcuts.

7.2. Part 2: Brush selection

Press `B` to turn on the brush. Paint on the screen to select gaussians. They turn yellow. The centre of each gaussian is projected to a pixel $\mathbf{p}$ (Section 3.1).

Capsule test. A stroke from $\mathbf{a}$ to $\mathbf{b}$ is a capsule of radius r. A gaussian is selected if

$$\|\mathbf{p}-\mathbf{a}-\tau(\mathbf{b}-\mathbf{a})\| \le r, \quad \tau = \operatorname{clip}\frac{(\mathbf{p}-\mathbf{a})\cdot(\mathbf{b}-\mathbf{a})}{\|\mathbf{b}-\mathbf{a}\|^2} \in [0,1]. \tag{28}$$

That is, the distance from $\mathbf{p}$ to the segment $\mathbf{ab}$ is at most r.

Properties. The brush tests only the gaussian centre and does not test depth. So it selects through the scene, like SuperSplat [3]. For speed, the brush rejects a whole block of 1024 gaussians when the bounding box of the block is outside the stroke.

7.3. Part 3: Soft delete and undo

Each gaussian has one state byte on the GPU with two bits: *selected* and *deleted*. The `Delete` command only sets the "deleted" bit. The shader skips these gaussians. The original data does not change. Each delete is one command in `History` (Command pattern [8], at most 20 steps). `Ctrl+Z` restores the last command.

7.4. Part 4: Interface

The ImGui panel on the left controls layers, transforms, selection, camera mode and lighting. The HUD shows the crosshair, the clock, the position and the brush circle. Table 3 lists the main keys.

Table 3. Main controls.

Key / mouse	Function
Left / middle drag	Orbit / pan
Right drag + `WASD`	Look around, fly (`Ctrl` = fast)
`M`, `F5`, `C`, `V`	Fly/walk; third person; sit; wave
`F`, `G`	Frame the scene; show/hide grid
`B`, `[`, `]`	Brush; brush size
`Delete`, `Ctrl+Z`	Delete / undo
`Ctrl+O/I/S`	Open / add layer / save

8. Conclusion

Splat Editor can read, display, edit and save 3DGS files. The program uses all six technique groups of the course. Table 4 shows where each part is in the source code.

Table 4. Main source files (in `splat_editor/`).

Part	File
PLY read/write	`io/ply.py`, `io/codec.py`
Gaussian rendering	`render/splat_renderer.py`, `render/sorter.py`, `render/shaders/splat_prep.vert`
T1	`core/layer.py`, `core/transform.py`, `core/avatar_pose.py`
T2	`core/math3d.py`, `render/camera.py`
T3	`render/avatar_renderer.py`
T4	`core/sh.py`, `core/lighting.py`, `render/shaders/shadow.vert`
T5	`core/physics.py`, `core/avatar_pose.py`
T6	`editor/brush.py`, `core/history.py`, `ui/`

References

[1] J. L. Schönberger, J.-M. Frahm, Structure-from-motion revisited, in: CVPR, 2016, pp. 4104–4113.
[2] B. Kerbl, G. Kopanas, T. Leimkühler, G. Drettakis, 3D Gaussian splatting for real-time radiance field rendering, ACM Trans. Graph. 42(4) (2023) 139.
[3] PlayCanvas, SuperSplat: an open-source 3D Gaussian Splat editor, https://github.com/playcanvas/supersplat.
[4] M. Zwicker, H. Pfister, J. van Baar, M. Gross, EWA splatting, IEEE Trans. Vis. Comput. Graph. 8(3) (2002) 223–238.
[5] B. T. Phong, Illumination for computer generated pictures, Commun. ACM 18(6) (1975) 311–317.
[6] L. Williams, Casting curved shadows on curved surfaces, in: SIGGRAPH, 1978, pp. 270–274.
[7] W. T. Reeves, D. H. Salesin, R. L. Cook, Rendering antialiased shadows with depth maps, in: SIGGRAPH, 1987, pp. 283–291.
[8] E. Gamma, R. Helm, R. Johnson, J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software, Addison-Wesley, 1994.